Table Structure Example:

Let's assume we have the following two tables:

1. `employees` table:

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    hire_date DATE,
    department_id INT,
    salary DECIMAL(10, 2)
);
```

2. `departments` table:

```
CREATE TABLE departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(100)
);
```

INSERT INTO departments (department_id, department_name)

VALUES

(1, 'Sales'),

(2, 'Marketing'),

(3, 'Engineering'),

(4, 'HR'),

(5, 'Finance');

INSERT INTO employees (employee_id, first_name, last_name, hire_date, department_id, salary) VALUES

(1, 'John', 'Doe', '2015-06-23', 1, 55000.00),

(2, 'Jane', 'Smith', '2018-02-10', 2, 62000.00),

(3, 'Samuel', 'Adams', '2012-11-04', 3, 90000.00),

(4, 'Emily', 'Clark', '2020-03-15', 1, 45000.00),

(5, 'Daniel', 'Harris', '2016-07-19', 4, 49000.00),

(6, 'Rachel', 'Baker', '2019-10-01', 3, 95000.00),

(7, 'Paul', 'Jones', '2017-09-13', 2, 55000.00),

(8, 'Sophia', 'Taylor', '2014-12-21', 5, 73000.00),

(9, 'Michael', 'Lee', '2011-08-14', 4, 47000.00),

(10, 'Olivia', 'King', '2022-01-30', 3, 98000.00);

- To concat the first_name and last_name of employees into a single column named full_name.

Sol: SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;

```
mysql>  SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM
+---------------+
| full_name     |
+---------------+
| John Doe      |
| Jane Smith    |
| Samuel Adams  |
| Emily Clark   |
| Daniel Harris |
| Rachel Baker  |
| Paul Jones    |
| Sophia Taylor |
| Michael Lee   |
| Olivia King   |
+---------------+
10 rows in set (0.00 sec)
```

- Query that retrieves the first and last names of employees, as well as their department names by using a subquery inside the SELECT statement.
- Query counts the number of employees in each department.

Sol:

SELECT department_id, count(*) AS employee_count

FROM employees

GROUP BY department_id;

```
mysql> SELECT department_id, count(*) AS employee_count
    -> FROM employees
    -> GROUP BY department_id;
+---------------+----------------+
| department_id | employee_count |
+---------------+----------------+
|             1 |              2 |
|             2 |              2 |
|             3 |              3 |
|             4 |              2 |
|             5 |              1 |
+---------------+----------------+
5 rows in set (0.00 sec)
```

- Query to lists all departments and counts the number of employees in each department .
- SOL:
- SELECT d.department_name, COUNT(e.employee_id) AS employee_count
- FROM departments d LEFT JOIN employees e ON d.department_id =e.department_id
- GROUP BY d.department_name;

```
+-----------------+----------------+
| department_name | employee_count |
+-----------------+----------------+
| Sales           |              2 |
| Marketing       |              2 |
| Engineering     |              3 |
| HR              |              2 |
| Finance         |              1 |
+-----------------+----------------+
5 rows in set (0.00 sec)
```

- To find employees with salary greater than average salary of their department.

Sol: SELECT e.first_name, e.last_name, e.salary, e.department_id

```
FROM employees e
WHERE e.salary > (
                    SELECT AVG(e2.salary)
                     FROM employees e2
                     WHERE e2.department_id = e.department_id
                     );
```

```
+------------+-----------+----------+---------------+
| first_name | last_name | salary   | department_id |
+------------+-----------+----------+---------------+
| John       | Doe       | 55000.00 |             1 |
| Jane       | Smith     | 62000.00 |             2 |
| Daniel     | Harris    | 49000.00 |             4 |
| Rachel     | Baker     | 95000.00 |             3 |
| Olivia     | King      | 98000.00 |             3 |
+------------+-----------+----------+---------------+
5 rows in set (0.00 sec)
```

- To get all employee names in each department.

Sol:

```
SELECT d.department_name,
  CONCAT(e.first_name, ' ', e.last_name) AS employee_name
   FROM employees e
   JOIN departments d
   ON e.department_id = d.department_id
   ORDER BY d.department_name;
```

```
+-----------------+---------------+
| department_name | employee_name |
+-----------------+---------------+
| Engineering     | Samuel Adams  |
| Engineering     | Rachel Baker  |
| Engineering     | Olivia King   |
| Finance         | Sophia Taylor |
| HR              | Daniel Harris |
| HR              | Michael Lee   |
| Marketing       | Jane Smith    |
| Marketing       | Paul Jones    |
| Sales           | John Doe      |
| Sales           | Emily Clark   |
+-----------------+---------------+
10 rows in set (0.00 sec)
```

- • Subquery to find the employee with the highest salary in each department.

Sol: SELECT d.department_name, MAX(e.salary) AS max_salary
FROM departments d
JOIN employees e
ON d.department_id = e.department_id
GROUP BY d.department_name;

```
+-----------------+------------+
| department_name | max_salary |
+-----------------+------------+
| Sales           |   55000.00 |
| Marketing       |   62000.00 |
| Engineering     |   98000.00 |
| HR              |   49000.00 |
| Finance         |   73000.00 |
+-----------------+------------+
5 rows in set (0.00 sec)
```

- • Group employees by hire year and calculate the total salary for each year

Sol:

```
SELECT YEAR(hire_date) AS hire_year,  SUM(salary) AS total_salary
FROM employees
GROUP BY YEAR(hire_date);
```

```
+-----------+--------------+
| hire_year | total_salary |
+-----------+--------------+
|      2015 |     55000.00 |
|      2018 |     62000.00 |
|      2012 |     90000.00 |
|      2020 |     45000.00 |
|      2016 |     49000.00 |
|      2019 |     95000.00 |
|      2017 |     55000.00 |
|      2014 |     73000.00 |
|      2011 |     47000.00 |
|      2022 |     98000.00 |
+-----------+--------------+
10 rows in set (0.00 sec)
```

- • Group departments and count the number of employees in each department

SOL:

```
SELECT d.department_name, COUNT(e.employee_id) AS employee_count
FROM departments d
LEFT JOIN employees e
ON d.department_id = e.department_id
GROUP BY d.department_name;
```

department_name	employee_count
Sales	2
Marketing	2
Engineering	3
HR	2
Finance	1

5 rows in set (0.00 sec)

- Find the highest salary in each department

SOL:

```
SELECT department_id, MAX(salary) AS highest_salary
FROM employees
GROUP BY department_id;
```

department_id	highest_salary
1	55000.00
2	62000.00
3	98000.00
4	49000.00
5	73000.00

5 rows in set (0.00 sec)

- Group employees by department and show the average salary in each department.

SOL:

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id;
```

```
+---------------+--------------+
| department_id | avg_salary   |
+---------------+--------------+
|             1 | 50000.000000 |
|             2 | 58500.000000 |
|             3 | 94333.333333 |
|             4 | 48000.000000 |
|             5 | 73000.000000 |
+---------------+--------------+
5 rows in set (0.00 sec)
```

- Find departments with more than 5 employees

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

- List departments where the average salary is greater than 50,000.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 50000;
```

```
+---------------+--------------+
| department_id | avg_salary   |
+---------------+--------------+
|             2 | 58500.000000 |
|             3 | 94333.333333 |
|             5 | 73000.000000 |
+---------------+--------------+
3 rows in set (0.00 sec)
```

- List employees who earn more than 60,000 and belong to a department with more than 3 employees.

```
SELECT department_id
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 3;
```

- Show departments where there is more than one employee with a salary over 100,000.

```
SELECT d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id
WHERE e.salary > 100000
GROUP BY d.department_name
HAVING COUNT(e.employee_id) > 1;
```

- Delete an employee with a specific employee_id (e.g., 5)

```
DELETE FROM employees
WHERE employee_id = 5;
```

-

```
mysql> DELETE FROM employees
    -> WHERE employee_id = 5;
Query OK, 1 row affected (0.01 sec)
```

MySQL Update Table exercises

- • Write a SQL statement to change the email column of employees table with 'not available' for all employees.

```
ALTER TABLE employees ADD email VARCHAR(100);
```

```
+--------+
| email  |
+--------+
| NULL   |
| NULL   |
| NULL   |
| NULL   |
| NULL   |
| NULL   |
| NULL   |
| NULL   |
| NULL   |
+--------+
9 rows in set (0.01 sec)
```

- • Write a SQL statement to change the email and commission_pct column of employees table with 'not available' and 0.10 for all employees.

```
ALTER TABLE employees
ADD commission_pct DECIMAL(5,2);
```

```
+-------------+------------+-----------+------------+---------------+
| employee_id | first_name | last_name | hire_date  | department_id |
+-------------+------------+-----------+------------+---------------+
|           1 | John       | Doe       | 2015-06-23 |             1 |
|           2 | Jane       | Smith     | 2018-02-10 |             2 |
|           3 | Samuel     | Adams     | 2012-11-04 |             3 |
|           4 | Emily      | Clark     | 2020-03-15 |             1 |
|           6 | Rachel     | Baker     | 2019-10-01 |             3 |
|           7 | Paul       | Jones     | 2017-09-13 |             2 |
|           8 | Sophia     | Taylor    | 2014-12-21 |             5 |
|           9 | Michael    | Lee       | 2011-08-14 |             4 |
|          10 | Olivia     | King      | 2022-01-30 |             3 |
+-------------+------------+-----------+------------+---------------+
9 rows in set (0.00 sec)
```

3. Write a SQL statement to change the email and commission_pct column of employees table with 'not available' and 0.10 for those employees whose department_id is 110.

```
UPDATE employees
SET email = 'not available',
   commission_pct = 0.10
WHERE department_id = 110;
```

```
mysql> UPDATE employees
    -> SET email = 'not available',
    ->     commission_pct = 0.10
    -> WHERE department_id = 110;
Query OK, 0 rows affected (0.01 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

- • Write a SQL statement to change the email column of employees table with 'not available' for those employees whose department_id is 80 and gets a commission is less than .20%

```
UPDATE employees
SET email = 'not available'
WHERE department_id = 80
AND commission_pct < 0.20;
```

```
mysql> UPDATE employees
    -> SET email = 'not available'
    -> WHERE department_id = 80
    -> AND commission_pct < 0.20;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

- Write a SQL statement to change the email column of employees table with 'not available' for those employees who belongs to the 'Accouning' department.

```
UPDATE employees e
JOIN departments d
ON e.department_id = d.department_id
SET e.email = 'not available'
WHERE d.department_name = 'Accounting';
```

- Write a SQL statement to change salary of employee to 8000 whose ID is 105, if the existing salary is less than 5000.

```
UPDATE employees
SET salary = 8000
WHERE employee_id = 105
AND salary < 5000;
```

- Write a SQL statement to change job ID of employee which ID is 118, to SH_CLERK if the employee belongs to department, which ID is 30 and the existing job ID does not start with SH.

Sol:
```
UPDATE employees
SET job_id = 'SH_CLERK'
WHERE employee_id = 118
AND department_id = 30
AND job_id NOT LIKE 'SH%';
```

MySQL Alter Table exercises

- Write a SQL statement to rename the table countries to country_new.

```
RENAME TABLE countries TO country_new;
```

- Write a SQL statement to add a column region_id to the table locations.

```
ALTER TABLE locations
ADD region_id INT;
```

- Write a SQL statement to add a columns ID as the first column of the table locations.
  ALTER TABLE locations
  ADD ID INT FIRST;

- Write a SQL statement to add a column region_id after state_province to the table locations.
  ALTER TABLE locations
  ADD COLUMN region_id INT AFTER state_province;

- Write a SQL statement change the data type of the column country_id to integer in the table locations.
  Sol: ALTER TABLE locations
  MODIFY COLUMN country_id INT;

- Write a SQL statement to drop the column city from the table locations.
  ALTER TABLE locations
  DROP COLUMN city;

7. Write a SQL statement to change the name of the column state_province to state, keeping the data type and size same.
ALTER TABLE locations
RENAME COLUMN state_province TO state;
8. Write a SQL statement to add a primary key for the columns location_id in the locations table. Here is the sample table employees.
Sample table: employees
ALTER TABLE locations
ADD PRIMARY KEY (location_id);

9. Write a SQL statement to add a primary key for a combination of columns location_id and country_id.
ALTER TABLE locations
ADD PRIMARY KEY (location_id, country_id);

10. Write a SQL statement to drop the existing primary from the table locations on a combination of columns location_id and country_id.
ALTER TABLE locations
DROP PRIMARY KEY;

Basic SELECT Statement Exercises

Sample table: employees

1. Write a query to display the names (first_name, last_name) using alias name “First Name", "Last Name"

```
SELECT first_name AS "First Name",
 last_name  AS "Last Name"
 FROM employees;
```

```
+------------+-----------+
| First Name | Last Name |
+------------+-----------+
| John       | Doe       |
| Jane       | Smith     |
| Samuel     | Adams     |
| Emily      | Clark     |
| Rachel     | Baker     |
| Paul       | Jones     |
| Sophia     | Taylor    |
| Michael    | Lee       |
| Olivia     | King      |
+------------+-----------+
9 rows in set (0.00 sec)
```

2. Write a query to get unique department ID from employee table.

```
SELECT DISTINCT department_id
 FROM departments;
```

```
+---------------+
| department_id |
+---------------+
|             1 |
|             2 |
|             3 |
|             4 |
|             5 |
+---------------+
5 rows in set (0.00 sec)
```

3. Write a query to get all employee details from the employee table order by first name,

descending.

```
SELECT *
 FROM employees
ORDER BY first_name DESC;
```

employee_id	first_name	last_name	hire_date	department_id
8	Sophia	Taylor	2014-12-21	5
3	Samuel	Adams	2012-11-04	3
6	Rachel	Baker	2019-10-01	3
7	Paul	Jones	2017-09-13	2
10	Olivia	King	2022-01-30	3
9	Michael	Lee	2011-08-14	4
1	John	Doe	2015-06-23	1
2	Jane	Smith	2018-02-10	2
4	Emily	Clark	2020-03-15	1

9 rows in set (0.00 sec)

4. Write a query to get the names (first_name, last_name), salary, PF of all the employees (PF is calculated as 12% of salary).

```
SELECT first_name,
    last_name,
    salary,
    salary * 0.12 AS PF
FROM employees;
```

first_name	last_name	salary	PF
John	Doe	55000.00	6600.0000
Jane	Smith	62000.00	7440.0000
Samuel	Adams	90000.00	10800.0000
Emily	Clark	45000.00	5400.0000
Rachel	Baker	95000.00	11400.0000
Paul	Jones	55000.00	6600.0000
Sophia	Taylor	73000.00	8760.0000
Michael	Lee	47000.00	5640.0000
Olivia	King	98000.00	11760.0000

9 rows in set (0.00 sec)

5. Write a query to get the employee ID, names (first_name, last_name), salary in ascending order of salary.

```
SELECT employee_id,
      first_name,
      last_name,
      salary
FROM employees
ORDER BY salary ASC;
```

```
+-------------+------------+-----------+----------+
| employee_id | first_name | last_name | salary   |
+-------------+------------+-----------+----------+
|           4 | Emily      | Clark     | 45000.00 |
|           9 | Michael    | Lee       | 47000.00 |
|           1 | John       | Doe       | 55000.00 |
|           7 | Paul       | Jones     | 55000.00 |
|           2 | Jane       | Smith     | 62000.00 |
|           8 | Sophia     | Taylor    | 73000.00 |
|           3 | Samuel     | Adams     | 90000.00 |
|           6 | Rachel     | Baker     | 95000.00 |
|          10 | Olivia     | King      | 98000.00 |
+-------------+------------+-----------+----------+
9 rows in set (0.00 sec)
```

6. Write a query to get the total salaries payable to employees.

```
SELECT SUM(salary) AS total_salaries
FROM employees;
```

```
+---------------+
| total_salaries |
+---------------+
|      620000.00 |
+---------------+
1 row in set (0.00 sec)
```

7. Write a query to get the maximum and minimum salary from employees table.

```
SELECT MAX(salary) AS max_salary,
 MIN(salary) AS min_salary
 FROM employees;
```

```
| max_salary | min_salary |
+------------+------------+
|   98000.00 |   45000.00 |
+------------+------------+
1 row in set (0.00 sec)
```

8. Write a query to get the average salary and number of employees in the employees table.

```
SELECT AVG(salary) AS average_salary,
COUNT(*) AS number_of_employees
From employees;
```

```
+----------------+---------------------+
| average_salary | number_of_employees |
+----------------+---------------------+
|   68888.888889 |                   9 |
+----------------+---------------------+
1 row in set (0.00 sec)
```

9. Write a query to get the number of employees working with the company.

```
SELECT COUNT(*) AS number_of_employees
FROM employees;
```

```
+---------------------+
| number_of_employees |
+---------------------+
|                   9 |
+---------------------+
1 row in set (0.01 sec)
```

MySQL Aggregate Functions and Group by- Exercises

Sample table: employees

1. Write a query to list the number of jobs available in the employees table.

```
SELECT COUNT(DISTINCT department_name) AS total_jobs
FROM departments;
```

```
+-------------+
| total_jobs |
+-------------+
|          5 |
+-------------+
1 row in set (0.00 sec)
```

2. Write a query to get the total salaries payable to employees.

```
SELECT SUM(salary) AS total_salaries
FROM employees;
```

```
+-----------------+
| total_salaries |
+-----------------+
|      620000.00 |
+-----------------+
1 row in set (0.00 sec)
```

3. Write a query to get the minimum salary from employees table.

```
  SELECT MIN(salary) AS minimum_salary
FROM employees;
```

```
+----------------+
| minimum_salary |
+----------------+
|       45000.00 |
+----------------+
1 row in set (0.00 sec)
```

4. Write a query to get the maximum salary of an employee working as a Programmer.

```
SELECT MAX(e.salary) AS max_salary
FROM employees e
JOIN departments d
ON e.department_id = d.department_id
WHERE d.department_name = 'Engineering';
```

```
+-------------+
| max_salary |
+-------------+
|   98000.00 |
+-------------+
1 row in set (0.00 sec)
```

5. Write a query to get the average salary and number of employees working the department 90.

```
   SELECT AVG(salary) AS average_salary,
      COUNT(*) AS number_of_employees
FROM employees
WHERE department_id = 90;
```

```
+----------------+----------------------+
| average_salary | number_of_employees |
+----------------+----------------------+
|           NULL |                    0 |
+----------------+----------------------+
1 row in set (0.00 sec)
```

6. Write a query to get the highest, lowest, sum, and average salary of all employees.

```
SELECT MAX(salary) AS highest_salary,
      MIN(salary) AS lowest_salary,
      SUM(salary) AS total_salary,
      AVG(salary) AS average_salary
FROM employees;
```

```
+----------------+---------------+--------------+-----------------+
| highest_salary | lowest_salary | total_salary | average_salary |
+----------------+---------------+--------------+-----------------+
|       98000.00 |      45000.00 |    620000.00 |   68888.888889 |
+----------------+---------------+--------------+-----------------+
1 row in set (0.00 sec)
```

7. Write a query to get the number of employees with the same job.

```
SELECT department_id,
      COUNT(*) AS number_of_employees
FROM employees
GROUP BY department_id;
```

```
+---------------+---------------------+
| department_id | number_of_employees |
+---------------+---------------------+
|             1 |                   2 |
|             2 |                   2 |
|             3 |                   3 |
|             5 |                   1 |
|             4 |                   1 |
+---------------+---------------------+
5 rows in set (0.00 sec)
```

MySQL Restricting and Sorting Data – Exercises

Sample table: employees

1. Write a query to display the names (first_name, last_name) and salary for all employees whose salary is not in the range 10,000 through 15,000.

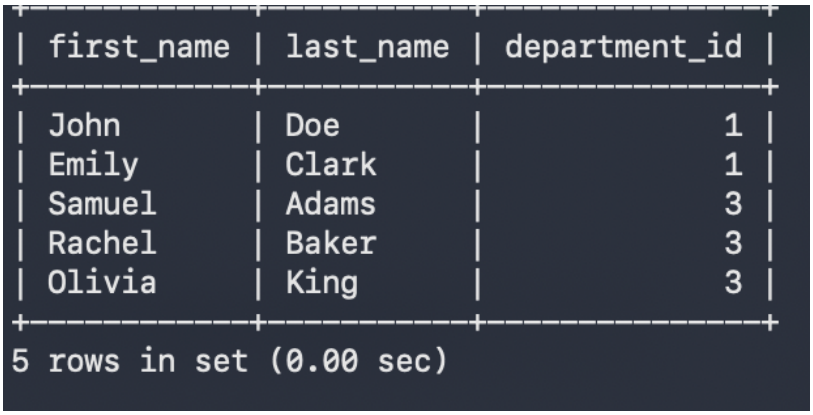
```
SELECT first_name, last_name, salary
FROM employees
WHERE salary NOT BETWEEN 10000 AND 15000;
```

```
+------------+-----------+----------+
| first_name | last_name | salary   |
+------------+-----------+----------+
| John       | Doe       | 55000.00 |
| Jane       | Smith     | 62000.00 |
| Samuel     | Adams     | 90000.00 |
| Emily      | Clark     | 45000.00 |
| Rachel     | Baker     | 95000.00 |
| Paul       | Jones     | 55000.00 |
| Sophia     | Taylor    | 73000.00 |
| Michael    | Lee       | 47000.00 |
| Olivia     | King      | 98000.00 |
+------------+-----------+----------+
9 rows in set (0.00 sec)
```

2. Write a query to display the names (first_name, last_name) and department ID of all employees in departments 3 or 1in ascending alphabetical order by department ID.

```
SELECT first_name, last_name, department_id
```

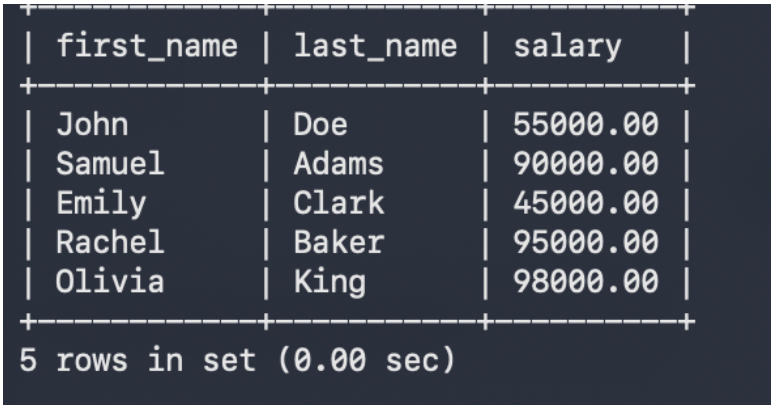
FROM employees
WHERE department_id IN (1, 3)
ORDER BY department_id ASC;



first_name	last_name	department_id
John	Doe	1
Emily	Clark	1
Samuel	Adams	3
Rachel	Baker	3
Olivia	King	3

5 rows in set (0.00 sec)

3. Write a query to display the names (first_name, last_name) and salary for all employees whose salary is not in the range 10,000 through 15,000 and are in department 3 or 1.



first_name	last_name	salary
John	Doe	55000.00
Samuel	Adams	90000.00
Emily	Clark	45000.00
Rachel	Baker	95000.00
Olivia	King	98000.00

5 rows in set (0.00 sec)

SELECT first_name, last_name, salary
FROM employees
WHERE salary NOT BETWEEN 10000 AND 15000
AND department_id IN (1, 3);

4. Write a query to display the names (first_name, last_name) and hire date for all employees who were hired in `2015-06-2`.

SELECT first_name, last_name, hire_date
FROM employees
WHERE YEAR(hire_date) = `2015`;

```
+------------+-----------+------------+
| first_name | last_name | hire_date  |
+------------+-----------+------------+
| John       | Doe       | 2015-06-23 |
+------------+-----------+------------+
1 row in set (0.00 sec)
```

5. Write a query to display the first_name of all employees who have both "b" and "c" in their first name.

6. Write a query to display the last name, job, and salary for all employees whose job is that of a Programmer or a Shipping Clerk, and whose salary is not equal to 4,500, 10,000, or 15,000.

7. Write a query to display the last names of employees whose names have exactly 6 characters.

```
SELECT last_name
FROM employees
WHERE LENGTH(last_name) = 6;
```

```
+-----------+
| last_name |
+-----------+
| Taylor    |
+-----------+
1 row in set (0.01 sec)
```

8. Write a query to display the last names of employees having 'e' as the third character.

```
SELECT last_name
FROM employees
WHERE last_name LIKE '__e%';
```

```
+-----------+
| last_name |
+-----------+
| Doe       |
| Lee       |
+-----------+
2 rows in set (0.00 sec)
```

9. Write a query to display the jobs/designations available in the employees table.

10. Write a query to display the names (first_name, last_name), salary and PF (15% of salary) of all employees.

```
SELECT first_name, last_name, salary,
      salary * 0.15 AS PF
FROM employees;
```

```
+------------+-----------+----------+------------+
| first_name | last_name | salary   | PF         |
+------------+-----------+----------+------------+
| John       | Doe       | 55000.00 |  8250.0000 |
| Jane       | Smith     | 62000.00 |  9300.0000 |
| Samuel     | Adams     | 90000.00 | 13500.0000 |
| Emily      | Clark     | 45000.00 |  6750.0000 |
| Rachel     | Baker     | 95000.00 | 14250.0000 |
| Paul       | Jones     | 55000.00 |  8250.0000 |
| Sophia     | Taylor    | 73000.00 | 10950.0000 |
| Michael    | Lee       | 47000.00 |  7050.0000 |
| Olivia     | King      | 98000.00 | 14700.0000 |
+------------+-----------+----------+------------+
9 rows in set (0.00 sec)
```

11. Write a query to select all record from employees where last name in 'BLAKE', 'SCOTT', 'KING' and 'FORD'.

```
SELECT *
FROM employees
WHERE last_name IN ('BLAKE', 'SCOTT', 'KING', 'FORD');
```

```
+-------------+------------+-----------+-------------+---------------
| employee_id | first_name | last_name | hire_date   | department_id
+-------------+------------+-----------+-------------+---------------
|          10 | Olivia     | King      | 2022-01-30 |             3
+-------------+------------+-----------+-------------+---------------
1 row in set (0.00 sec)
```

MySQL Sub Queries Exercises

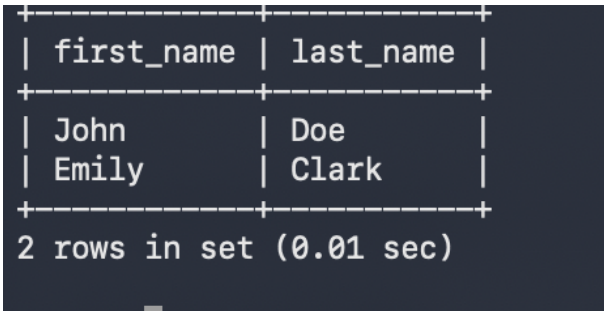
1. Write a query to find the names (first_name, last_name) and the salaries of the employees who have a higher salary than the employee whose last_name='lee'.

```
SELECT FIRST_NAME, LAST_NAME, SALARY
FROM employees
WHERE salary > (SELECT salary FROM employees WHERE last_name = 'lee' );
```

```
+------------+-----------+----------+
| first_name | last_name | salary   |
+------------+-----------+----------+
| John       | Doe       | 55000.00 |
| Jane       | Smith     | 62000.00 |
| Samuel     | Adams     | 90000.00 |
| Rachel     | Baker     | 95000.00 |
| Paul       | Jones     | 55000.00 |
| Sophia     | Taylor    | 73000.00 |
| Olivia     | King      | 98000.00 |
+------------+-----------+----------+
7 rows in set (0.00 sec)
```

2. Write a query to find the names (first_name, last_name) of all employees who works in the sales department.

```
SELECT e.first_name, e.last_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id
WHERE d.department_name = 'Sales';
```



```
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| John       | Doe       |
| Emily      | Clark     |
+------------+-----------+
2 rows in set (0.01 sec)
```

3.Write a query to find the names (first_name, last_name) of the employees who have a manager and work for a department based in the United States.
Hint : Write single-row and multiple-row subqueries
Sample table: employees
Sample table: departments
Sample table: locations

```
SELECT first_name, last_name
FROM employees
WHERE manager_id IS NOT NULL
AND department_id IN (
   SELECT department_id
   FROM departments
   WHERE location_id IN (
      SELECT location_id
      FROM locations
      WHERE country_id = 'US'
   )
);
```

4. Write a query to find the names (first_name, last_name) of the employees who are managers.

5. Write a query to find the names (first_name, last_name), the salary of the employees whose salary is greater than the average salary.

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (
   SELECT AVG(salary)
   FROM employees
);
```

```
+------------+-----------+----------+
| first_name | last_name | salary   |
+------------+-----------+----------+
| Samuel     | Adams     | 90000.00 |
| Rachel     | Baker     | 95000.00 |
| Sophia     | Taylor    | 73000.00 |
| Olivia     | King      | 98000.00 |
+------------+-----------+----------+
4 rows in set (0.00 sec)
```

6. Write a query to find the names (first_name, last_name), the salary of the employees whose salary is equal to the minimum salary for their job grade.
Sample table: employees
Sample table: jobs

```
SELECT e.first_name, e.last_name, e.salary
FROM employees e
WHERE e.salary = (
   SELECT j.min_salary
   FROM jobs j
   WHERE j.job_id = e.job_id
);
```

7. Write a query to find the names (first_name, last_name), the salary of the employees who earn more than the average salary and who works in any of the marketing departments.
Sample table: employees
Sample table: departments

```
SELECT first_name, last_name, salary
FROM employees
WHERE department_id IN (SELECT department_id
                        FROM departments
                        WHERE department_name LIKE 'IT%') AND salary >
                                                            (SELECT
avg(salary)
                                                            FROM
employees);
```

8. Write a query to find the names (first_name, last_name), the salary of the employees who earn more than Mr. Brown.
Sample table: employees
Sample table: departments

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary >
(SELECT salary FROM employees WHERE last_name = 'Bell') ORDER BY first_name;
```

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (
   SELECT salary
   FROM employees
   WHERE last_name = 'Bro'wn
);
```

9. Write a query to find the names (first_name, last_name), the salary of the employees who earn the same salary as the minimum salary for all departments.
Sample table: employees
Sample table: departments

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary = (
   SELECT MIN(salary)
   FROM employees
);
```

```
+------------+-----------+----------+
| first_name | last_name | salary   |
+------------+-----------+----------+
| Emily      | Clark     | 45000.00 |
+------------+-----------+----------+
1 row in set (0.00 sec)
```

10. Write a query to find the names (first_name, last_name), the salary of the employees whose salary greater than the average salary of all departments.
Sample table: employees

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (
   SELECT AVG(salary)
```

```
   FROM employees
);
```

```
+------------+-----------+----------+
| first_name | last_name | salary   |
+------------+-----------+----------+
| Samuel     | Adams     | 90000.00 |
| Rachel     | Baker     | 95000.00 |
| Sophia     | Taylor    | 73000.00 |
| Olivia     | King      | 98000.00 |
+------------+-----------+----------+
4 rows in set (0.00 sec)
```

11. Write a query to find the names (first_name, last_name) and salary of the employees who earn a salary that is higher than the salary of all the Shipping Clerk (JOB_ID = 'SH_CLERK'). Sort the results of the salary of the lowest to highest.
Sample table: employees

12. Write a query to find the names (first_name, last_name) of the employees who are not supervisors.
Sample table: employees

13. Write a query to display the employee ID, first name, last names, and department names of all employees.
Sample table: employees
Sample table: departments

```
SELECT e.employee_id, e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;
```

employee_id	first_name	last_name	department_name
1	John	Doe	Sales
2	Jane	Smith	Marketing
3	Samuel	Adams	Engineering
4	Emily	Clark	Sales
6	Rachel	Baker	Engineering
7	Paul	Jones	Marketing
8	Sophia	Taylor	Finance
9	Michael	Lee	HR
10	Olivia	King	Engineering

9 rows in set (0.00 sec)

14. Write a query to display the employee ID, first name, last names, salary of all employees whose salary is above average for their departments.
Sample table: employees
Sample table: departments

```
SELECT e.employee_id, e.first_name, e.last_name, e.salary
FROM employees e
WHERE e.salary > (
   SELECT AVG(salary)
   FROM employees
   WHERE department_id = e.department_id
);
```

employee_id	first_name	last_name	salary
1	John	Doe	55000.00
2	Jane	Smith	62000.00
6	Rachel	Baker	95000.00
10	Olivia	King	98000.00

4 rows in set (0.01 sec)

15. Write a query to fetch even numbered records from employees table.
Sample table: employees

```
SELECT *
FROM employees
WHERE employee_id % 2 = 0;
```

employee_id	first_name	last_name	hire_date	department_id
2	Jane	Smith	2018-02-10	2
4	Emily	Clark	2020-03-15	1
6	Rachel	Baker	2019-10-01	3
8	Sophia	Taylor	2014-12-21	5
10	Olivia	King	2022-01-30	3

5 rows in set (0.00 sec)

16. Write a query to find the 5th maximum salary in the employees table.
Sample table: employees

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 4, 1;
```

salary
62000.00

1 row in set (0.01 sec)

17. Write a query to find the 4th minimum salary in the employees table.
Sample table: employees

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary ASC
LIMIT 3, 1;
```

salary
62000.00

1 row in set (0.00 sec)

18. Write a query to select last 10 records from a table.
Sample table: employees

```
SELECT *
FROM employees
ORDER BY employee_id DESC
LIMIT 10;
```

employee_id	first_name	last_name	hire_date	department_id
10	Olivia	King	2022-01-30	3
9	Michael	Lee	2011-08-14	4
8	Sophia	Taylor	2014-12-21	5
7	Paul	Jones	2017-09-13	2
6	Rachel	Baker	2019-10-01	3
4	Emily	Clark	2020-03-15	1
3	Samuel	Adams	2012-11-04	3
2	Jane	Smith	2018-02-10	2
1	John	Doe	2015-06-23	1

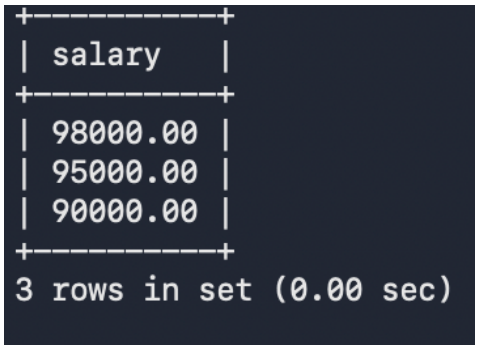
9 rows in set (0.00 sec)

19. Write a query to list department number, name for all the departments in which there are no employees in the department.
Sample table: employees
Sample table: departments

```
SELECT department_id, department_name
FROM departments
WHERE department_id NOT IN (
   SELECT department_id
   FROM employees
);
```

20. Write a query to get 3 maximum salaries.

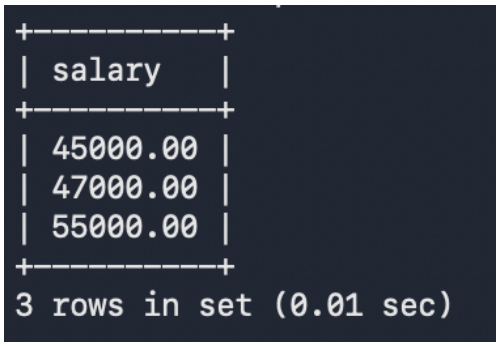
```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```



```
+----------+
| salary   |
+----------+
| 98000.00 |
| 95000.00 |
| 90000.00 |
+----------+
3 rows in set (0.00 sec)
```

21. Write a query to get 3 minimum salaries.

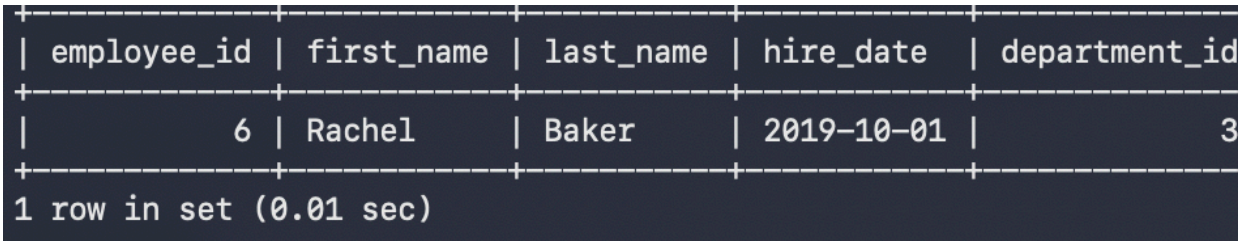
```
SELECT DISTINCT salary
FROM employees
ORDER BY salary ASC
LIMIT 3;
```



```
+----------+
| salary   |
+----------+
| 45000.00 |
| 47000.00 |
| 55000.00 |
+----------+
3 rows in set (0.01 sec)
```

22. Write a query to get nth max salaries of employees.

```
SELECT *
 FROM employees emp1
WHERE (1) =
                (SELECT COUNT(DISTINCT(emp2.salary))
                FROM employees emp2
                WHERE emp2.salary > emp1.salary);
```



```
+-------------+------------+-----------+------------+--------------
| employee_id | first_name | last_name | hire_date  | department_id
+-------------+------------+-----------+------------+--------------
|           6 | Rachel     | Baker     | 2019-10-01 |             3
+-------------+------------+-----------+------------+--------------
1 row in set (0.01 sec)
```

MySQL JOINS – Exercises

1. Write a query to find the addresses (location_id, street_address, city, state_province, country_name) of all the departments.
Hint : Use NATURAL JOIN.
Sample table : locations
Sample table : countries

```
  SELECT location_id,
      street_address,
      city,
      state_province,
      country_name
FROM departments
NATURAL JOIN locations
NATURAL JOIN countries;
```

2. Write a query to find the names (first_name, last name), department ID and name of all the employees.
Sample table : employees
Sample table : departments

```
SELECT e.first_name,
      e.last_name,
      e.department_id,
      d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;
```

3. Find the names (first_name, last_name), job, department number, and department name of the employees who work in London.
Sample table : departments
Sample table : locations

```
SELECT e.first_name,
      e.last_name,
      e.job_id,
      e.department_id,
      d.department_name
FROM employees e
JOIN departments d
   ON e.department_id = d.department_id
JOIN locations l
   ON d.location_id = l.location_id
WHERE l.city = 'London';
```

4. Write a query to find the employee id, name (last_name) along with their manager_id, manager name (last_name).
Sample table : employees

```
 SELECT e.employee_id,
       e.last_name AS employee_name,
       e.manager_id,
       m.last_name AS manager_name
FROM employees e
LEFT JOIN employees m
   ON e.manager_id = m.employee_id;
```

5. Find the names (first_name, last_name) and hire date of the employees who were hired after 'Jones'.
Sample table : employees

```
 SELECT first_name,
       last_name,
       hire_date
FROM employees
WHERE hire_date > (
   SELECT hire_date
   FROM employees
   WHERE last_name = 'Jones'
);
```

6. Write a query to get the department name and number of employees in the department.
Sample table : employees
Sample table : departments

```
  SELECT d.department_name,
       COUNT(e.employee_id) AS number_of_employees
FROM departments d
LEFT JOIN employees e
   ON d.department_id = e.department_id
GROUP BY d.department_name;
```

7. Find the employee ID, job title, number of days between ending date and starting date for all jobs in department 90 from job history.
Sample table : employees

```
 SELECT employee_id,
       job_id,
       (end_date - start_date) AS number_of_days
FROM job_history
WHERE department_id = 90;
```

8. Write a query to display the department ID, department name and manager first name.
Sample table : employees
Sample table : departments

```
SELECT d.department_id,
      d.department_name,
      e.first_name AS manager_first_name
FROM departments d
JOIN employees e
```

```
ON d.manager_id = e.employee_id;
```

9. Write a query to display the department name, manager name, and city.
Sample table : employees
Sample table : departments
Sample table : locations

```
SELECT d.department_name, e.first_name, l.city
FROM departments d
JOIN employees e
ON (d.manager_id = e.employee_id)
JOIN locations l USING (location_id);
```

10. Write a query to display the job title and average salary of employees.
Sample table : employees

```
SELECT job_title, AVG(salary)
FROM employees
NATURAL JOIN jobs
GROUP BY job_title;
```

11. Display job title, employee name, and the difference between salary of the employee and minimum salary for the job.
Sample table : employees

```
SELECT job_title, first_name, salary-min_salary 'Salary - Min_Salary'
FROM employees
NATURAL JOIN jobs;
```

12. Write a query to display the job history that were done by any employee who is currently drawing more than 10000 of salary.
Sample table : employees
Sample table : Job_history

```
SELECT jh.* FROM job_history jh
JOIN employees e
ON (jh.employee_id = e.employee_id)
WHERE salary > 10000;
```

13. Write a query to display department name, name (first_name, last_name), hire date, salary of the manager for all managers whose experience is more than 15 years.
Sample table : employees
Sample table : departments

```
SELECT first_name, last_name, hire_date, salary,
(DATEDIFF(now(), hire_date))/365 Experience
FROM departments d JOIN employees e
ON (d.manager_id = e.employee_id)
WHERE (DATEDIFF(now(), hire_date))/365>15;
```

Joins Exercises Solutions

1
```
SELECT location_id, street_address, city, state_province, country_name
FROM locations
NATURAL JOIN countries;
```
2
```
SELECT first_name, last_name, department_id, department_name
FROM employees
JOIN
departments
USING (department_id);
```
3
```
SELECT e.first_name, e.last_name, e.job_id, e.department_id, d.department_name
FROM employees e
 JOIN
 departments d ON (e.department_id = d.department_id)
 JOIN
 locations l ON (d.location_id = l.location_id)
 WHERE LOWER(l.city) = 'London';
```
4
```
SELECT e.employee_id 'Emp_Id', e.last_name 'Employee', m.employee_id 'Mgr_Id',
m.last_name 'Manager'
FROM employees e
join employees m
 ON (e.manager_id = m.employee_id);
```
5
```
SELECT e.first_name, e.last_name, e.hire_date
 FROM employees e
 JOIN
 employees davies
 ON (davies.last_name = 'Jones')
 WHERE davies.hire_date < e.hire_date;
```

6.
```
SELECT department_name AS 'Department Name',
COUNT(*) AS 'No of Employees'
FROM departments
INNER JOIN employees
ON employees.department_id = departments.department_id
GROUP BY departments.department_id, department_name
ORDER BY department_name;
```

7
```
SELECT employee_id, job_title, end_date-start_date Days FROM job_history
```

```
NATURAL JOIN jobs
WHERE department_id=90;
```
8
```
SELECT d.department_id, d.department_name, e.manager_id, e.first_name
FROM departments d
INNER JOIN employees e
ON (d.manager_id = e.employee_id);
```
9
```
SELECT d.department_name, e.first_name, l.city
FROM departments d
JOIN employees e
ON (d.manager_id = e.employee_id)
JOIN locations l USING (location_id);
```
10
```
SELECT job_title, AVG(salary)
FROM employees
NATURAL JOIN jobs
GROUP BY job_title;
```
11
```
SELECT job_title, first_name, salary-min_salary 'Salary - Min_Salary'
FROM employees
NATURAL JOIN jobs;
```
12
```
SELECT jh.* FROM job_history jh
JOIN employees e
ON (jh.employee_id = e.employee_id)
WHERE salary > 10000;
```
13.
```
SELECT first_name, last_name, hire_date, salary,
(DATEDIFF(now(), hire_date))/365 Experience
FROM departments d JOIN employees e
ON (d.manager_id = e.employee_id)
WHERE (DATEDIFF(now(), hire_date))/365>15;
```

MySQL Northwind database, Products table – Exercises

1. Write a query to get Product name and quantity/unit.

2. Write a query to get current Product list (Product ID and name).

3. Write a query to get discontinued Product list (Product ID and name).

4. Write a query to get most expense and least expensive Product list (name and unit price).

5. Write a query to get Product list (id, name, unit price) where current products cost less than $20.

6. Write a query to get Product list (id, name, unit price) where products cost between $15 and $25.

7. Write a query to get Product list (name, unit price) of above average price.

8. Write a query to get Product list (name, unit price) of ten most expensive products.

9. Write a query to count current and discontinued products.

10. Write a query to get Product list (name, units on order , units in stock) of stock is less than the quantity on order.

... More
Structure of 'northwind' database :

MySQL Northwind database, Products table – Solution

1
```
SELECT ProductName, QuantityPerUnit
FROM Products;
```
2
```
SELECT ProductID, ProductName
FROM Products
WHERE Discontinued = "False"
```

```
ORDER BY ProductName;
3
SELECT ProductID, ProductName
FROM Products
WHERE Discontinued = "True"
ORDER BY ProductName;
4
SELECT ProductName, UnitPrice
FROM Products
ORDER BY UnitPrice DESC;
5
SELECT ProductID, ProductName, UnitPrice
FROM Products
WHERE (((UnitPrice)<20) AND ((Discontinued)=False))
ORDER BY UnitPrice DESC;
6
SELECT ProductName, UnitPrice
FROM Products
WHERE (((UnitPrice)>=15 And (UnitPrice)<=25)
AND ((Products.Discontinued)=False))
ORDER BY Products.UnitPrice DESC;
7
SELECT DISTINCT ProductName, UnitPrice
FROM Products
WHERE UnitPrice > (SELECT avg(UnitPrice) FROM Products)
ORDER BY UnitPrice;
8
SELECT DISTINCT ProductName as Twenty_Most_Expensive_Products, UnitPrice
FROM Products AS a
WHERE 20 >= (SELECT COUNT(DISTINCT UnitPrice)
                FROM Products AS b
                WHERE b.UnitPrice >= a.UnitPrice)
ORDER BY UnitPrice desc;
9
SELECT Count(ProductName)
FROM Products
GROUP BY Discontinued;
10.
SELECT ProductName,  UnitsOnOrder , UnitsInStock
FROM Products
WHERE (((Discontinued)=False) AND ((UnitsInStock)<UnitsOnOrder));
```